CMP4011 – Big Data and Cloud Computing

Distributed Football Transfer Market Analytics and Player Profiling

Final Project Report

Team Members

Name	Code
Muhammad Amr Hassan Muhammad	9220728
Ali Bahr Hussein Yousef	9220494
Mostafa Mohammed Rabie	9211196
Mohammed Khaled Mohammed	9221285

Submitted to: Dr. Lydia Wahid & Eng. Omar Samir

---

1. Problem Description

Modern football clubs invest heavily in scouting, player acquisition, and squad planning, yet transfer decisions are often uncertain because player value depends on many interacting factors like age, performance, injuries, league level, and transfer history. This project analyses large-scale football data to answer business-oriented questions such as:

- What types of players share similar performance profiles?
- What transfer patterns occur between leagues, positions, and age groups?
- Which agents are most active with specific clubs?
- What nationality pairings are most common among teammates?
- What citizenships that mostly held by the same player ?
- Can we predict whether a player will transfer in the next transfer window?

The goal is to provide insights and recommendations useful for clubs, scouts, analysts, and football business managers through both descriptive and predictive analytics.

---

2. Dataset

We used the **Comprehensive Football Dataset** from Kaggle:
https://www.kaggle.com/datasets/xfkzujqjvx97n/football-datasets

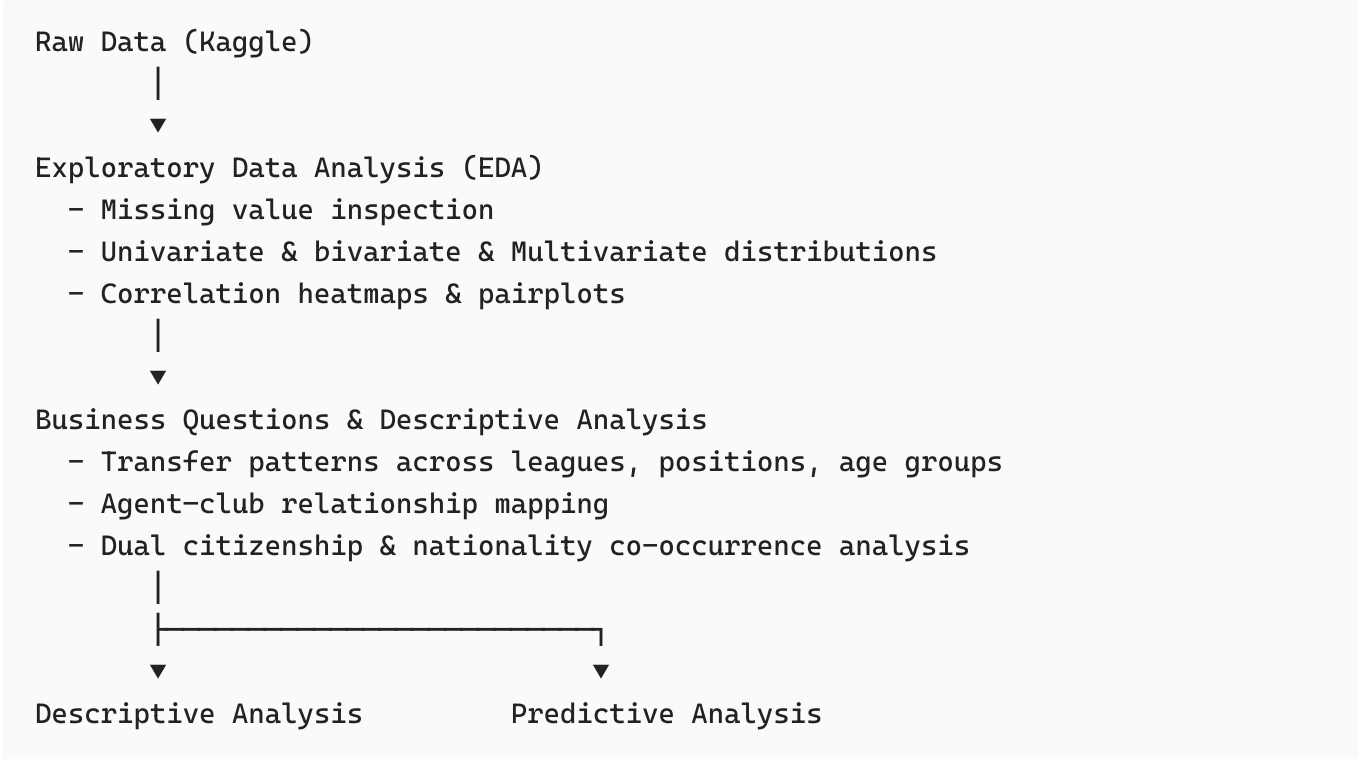
The dataset contains:

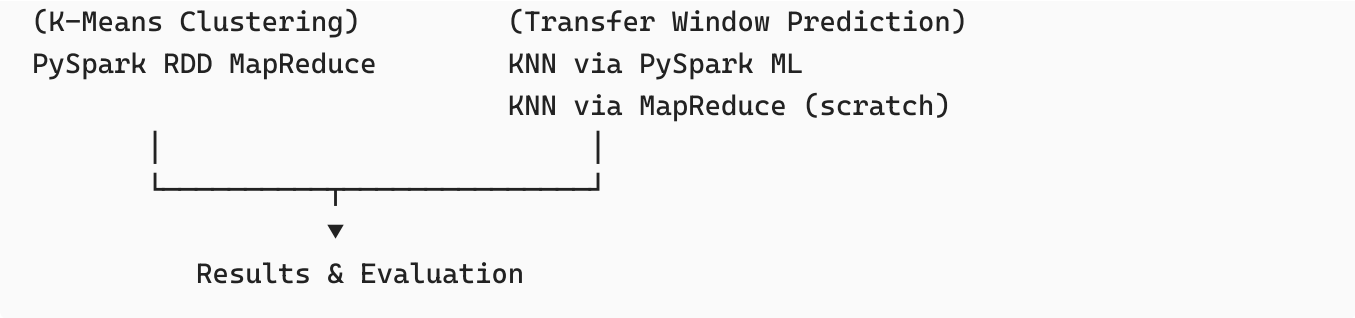
- **93,000+** players worldwide
- **2,200+** clubs across all major leagues
- **5.7M+** total records across 10 relational categories
- **902,000+** market valuations
- **1.9M+** player performance statistics
- **1.2M+** player transfer histories
- **144,000+** injury records
- **93,000+** national team appearances
- **1.3M+** teammate relationship records

The schema is fully relational, with tables including `player_profiles`, `player_performances`, `player_transfer_histories`, `player_market_values`, `team_details`, `team_competitions_seasons`, `player_national_team_performances`, and `player_teammates_played_with`.

---

3. Project Pipeline





---

4. Data Preprocessing

4.1 EDA Phase

A thorough Exploratory Data Analysis was conducted on each table in the dataset.

Player Profiles Table: The table contains 92,671 player records. High-cardinality features (URLs, slugs) were skipped in the univariate analysis. Key findings include:

- Height is normally distributed around 180 cm, with outliers near 0 (data quality issue).
- The most common position is Defender – Centre-Back, followed by Attack – Centre-Forward.
- Approximately 72% of players carry EU citizenship (`is_eu = True`).
- Several columns have very high missing rates (>60%): `fourth_club_url` , `third_club_url` , `second_club_url` , `date_of_death` , `contract_there_expires` , and `on_loan_from_*` fields.

Transfer History Table: Contains 1,101,440 records across 9 columns with near-zero missing values. The dominant transfer type is "Transfer" (≈844,000 records), followed by "Loan" and "Return from loan." Transfer fees and market values at transfer are heavily right-skewed with a strong correlation of 0.67 between the two.

Team Details Table: Contains 2,175 club entries. Italy has the most clubs in the dataset, followed by France and England. About 23% of records are missing competition-related fields.

Team Competitions Seasons Table: 196,378 records across seasons, with ~16% missing for competition-related columns.

Player Performances Table: 1,878,719 match-level records. The `minutes_played` column has a ~62% missing rate. All other columns are fully populated. Goals and assists show high correlation with appearances (nb_on_pitch), and minutes_played correlates moderately with goals_conceded and clean_sheets (goalkeeper performances).

4.2 Transfer Window Prediction, Preprocessing

For the predictive model, a dedicated preprocessing pipeline was applied:

- Only relevant columns were retained per table.
- Type coercion was performed on all numeric and date fields.
- Dates were parsed from both ISO format (`transfer_date`) and season-name format (`"YY/YY"`), unified into a single integer `season_year` .
  - Summer transfers (month ≠ January) were attributed to the next season.
  - Season names like `"24/25"` were resolved to `2025` via regex.
- Rows with null key identifiers were dropped; numeric nulls were filled with 0 where appropriate.
- Output was saved as CSV per table to `/PreProcessedData` for reuse.

---

5. Data Visualization & Business Questions

Business Question 1: What is the frequency of transfers between leagues?

Transfer heatmaps were generated across three time windows, last 5, 10, and 15 years, first for all competitions globally, then focused on the **top 30 most active league-to-league paths**.

Key observations from the top-30 heatmaps:

- Domestic recycling dominates: leagues such as Serie A, Campeonato Brasileiro Série A/B, Premier League, LaLiga, and Torneo Clausura show the darkest diagonal cells, meaning most transfers are internal to the same league ecosystem.
- The highest cross-league transfer volume over 15 years is Torneo Clausura → itself (2,458), followed by Campeonato Brasileiro Série A → itself (2,083) and Serie A → itself.
- Cross-border hotspots include transfers between Serie A and Serie B (Italy's top two divisions), Premier League and Championship (England), and Campeonato Brasileiro Série A/B.

Transfer patterns were also broken down by **age group** (<20, 20–23, 24–27, 28–31, 32+) and **position** (Attack, Defender, Goalkeeper, Midfield), revealing that younger players (20–23) show a concentration in youth academies and lower-division leagues, while veterans (32+) increasingly move to Torneo Clausura, USL Championship, and other lower-prestige leagues.

Business Question 2: Which agents deal with each club?

Agent–club relationship heatmaps were produced for the **Premier League**, **LaLiga**, and all leagues combined (top 30 agent–club paths).

Notable findings:

- **Wasserman** is the most active agent in the Premier League, appearing at Manchester United (985 interactions), Manchester City (399), Aston Villa (405), and Brentford FC (1,148).
- **CAA Stellar** has a strong presence at multiple Premier League clubs (5 deals each at several clubs).
- In LaLiga, **YOU FIRST** leads with 4 deals at Athletic Bilbao, while **Wasserman** and **CAA Stellar** remain active.
- Most clubs rely on a small number of well-connected agents, which suggests that scout and transfer networks are concentrated.

Business Question 3: What are the two most common citizenships a player holds?

A heatmap of the top 30 dual-citizenship pairs was produced. Key findings:

- **Italy – Argentina** is the most common dual-citizenship pair with 225 players.
- **France – Algeria** (156), **France – Morocco** (158), and **England – Ireland** (165) are the next most frequent.
- Many pairings reflect historical colonial ties or diaspora patterns (France–North Africa, England–Ireland, Italy–South America).

Business Question 4: What are the two most common nationalities among teammates?

Nationality co-occurrence heatmaps were generated both excluding and including same-country pairs.

- When excluding same-country pairing, **Italy–Spain** and **Brazil–Spain** emerge as the most frequent cross-nationality teammate pairs.
- Including all pairs, **Italy–Italy** (70,000+), **Spain–Spain**, and **Brazil–Brazil** dominate, reflecting the league composition of the dataset.
- Cross-national pairings are led by countries with large player exports: Brazil, France, England, Germany, and Portugal consistently appear in top co-occurrence pairs.

---

6. Predictive Analysis – Transfer Window Prediction

6.1 Problem Statement

Can we predict whether a player will transfer in the next transfer window?

This is framed as a binary classification task: the target variable is `1` if the player appeared in the transfer history for the target season, and `0` otherwise.

6.2 Feature Engineering

A rolling window aggregation approach was used to build features for each player–season pair:

- **Performance (last 3 seasons):** 12 metrics — goals, assists, minutes_played, yellow_cards, direct_red_cards, penalty_goals, clean_sheets, goals_conceded, subed_in, subed_out, nb_in_group, nb_on_pitch.
- **Performance (last 6 seasons):** Same 12 metrics with `_last6` suffix.
- **Injury (last 3 seasons):** days_missed, games_missed.
- **Injury (last 6 seasons):** Same 2 metrics.
- **National team (2 metrics):** international_matches, international_goals.

Total: 30 features. All features use data from strictly before the target season to prevent data leakage.

6.3 Normalization & Split

- **Normalization:** StandardScaler applied, fitting only on training data. Critical for KNN since unscaled features with large ranges dominate Euclidean distance.
- **Split strategy:** 70% training | 20% validation | 10% test, stratified to preserve class ratio across all splits.
- Scaled arrays were converted to PySpark DataFrames and RDDs for distributed processing. Training RDD was cached with `MEMORY_AND_DISK` persistence to avoid recomputation during repeated KNN scans.

6.4 Model 1: KNN using PySpark ML

- sklearn's `KNeighborsClassifier` was trained on the PySpark-preprocessed feature matrix.
- PySpark handled loading, scaling, and splitting, sklearn handled KNN fitting.
- **K = 3**, Euclidean distance metric.
- RDD repartitioned to 4 partitions for parallel distance computation.

Results:

Split	Accuracy	F1-Score
Validation	0.6408	0.2406
Test	0.6460	0.2410

Classification report on the test set (5,642 samples):

Class	Precision	Recall	F1
No Transfer	0.72	0.82	0.77
Transfer	0.31	0.20	0.24
Accuracy	—	—	0.65

The model struggles to detect actual transfers (minority class), a consequence of the strong class imbalance (~72% No Transfer vs ~28% Transfer).

6.5 Model 2: KNN with MapReduce (from scratch)

A pure PySpark RDD-based KNN implementation with no external ML library was developed. Training data was distributed across **5 partitions**.

Map stage (per partition): For each test point, every training point in the partition computed its Euclidean distance. Each partition emitted key-value pairs `(None, (distance, label))` and forwarded only local top-K neighbors to reduce shuffle size.

Reduce stage: Partition-local top-K lists were merged and re-sorted globally. Final top-K neighbors were selected across all partitions, and the predicted class was determined by majority vote.

Due to computational intensity, evaluation was performed on a subset: **1,000 train / 200 val / 100 test** samples.

Results:

Split	Accuracy	F1-Score
Validation	0.6200	0.2963
Test	0.6300	0.1395

Classification report on the 100-sample test set:

Class	Precision	Recall	F1
No Transfer	0.78	0.75	0.76
Transfer	0.13	0.15	0.14
Accuracy	—	—	0.63

Performance is broadly consistent with Model 1, confirming that the bottleneck is the class imbalance rather than the implementation choice.

---

7. Descriptive Analysis – Player Categorizing Model (K-Means via PySpark MapReduce)

7.1 Objective

Cluster players into **5 performance tiers** using unsupervised K-Means to identify archetypes useful for scouting and squad planning.

7.2 Features Used

Feature	What It Captures
nb_on_pitch_total	Total appearances
minutes_played_total	Time on pitch
goals_total	Club goals scored
assists_total	Club assists
penalty_goals_total	Penalty impact
clean_sheets_total	Defensive reliability
goals_conceded_total	Defensive pressure
yellow_cards_total	Discipline
national_matches_total	International exposure
national_goals_total	International impact

7.3 Pipeline

1. Raw data ingestion
2. Feature engineering (aggregate career totals per player)
3. StandardScaler normalization
4. TSV export for Spark input
5. PySpark RDD K-Means MapReduce (K = 5, max 20 iterations)
6. Cluster label assignment
7. Results visualization

7.4 MapReduce Architecture

The K-Means implementation (`kmeans_spark.py`) follows the classic MapReduce paradigm:

- **MAP:** Each player point is assigned to the nearest centroid using Euclidean distance. Centroids are broadcast as a read-only variable to all workers.
- **SHUFFLE:** PySpark groups all players by their assigned cluster ID via `groupByKey()`.
- **REDUCE:** A new centroid is computed as the mean of all players assigned to each cluster.
- **Initialization:** K-Means++ seeding — the first centroid is chosen randomly; each subsequent centroid is selected with probability proportional to its squared distance from existing centroids, improving convergence speed and cluster quality.
- **Convergence:** The algorithm terminates when the maximum centroid shift across all clusters falls below 1e-4 or after 20 iterations.

7.5 Results

The algorithm ran on **81,256 players** and produced the following cluster distribution:

Cluster Label	Count	%
Elite	1,605	2.0%
Star	4,284	5.3%
Reliable Player	2,452	3.0%
Squad Player	18,905	23.3%
Low Impact	54,010	66.5%

Cluster interpretation (from centroid heatmap):

- **Elite:** High appearances, assists, goals, and very high international matches (4.39σ) and international goals (5.26σ). The top global talent pool.
- **Star:** High goals, assists, and penalty impact. Marginally less international exposure than Elite, but strong all-round attackers.
- **Reliable Player:** Characterized by extremely high clean_sheets (4.87σ) and goals_conceded (4.93σ). This cluster captures goalkeepers and elite defenders.
- **Squad Player:** Moderate presence across all features. Solid rotation players with limited international exposure.
- **Low Impact:** Below-average across all dimensions. The majority of registered players worldwide fall here.

Clustering quality metrics:

Metric	Value
Silhouette Score	0.4532
Davies-Bouldin Index	1.1216
Calinski-Harabasz Index	30,651.0

A Silhouette Score of 0.45 indicates reasonable separation between clusters, though some overlap between Squad Player and Low Impact is expected given the continuous nature of the features.

---

8. Unsuccessful Trials

8.1 K-Means on Hadoop Streaming

An initial attempt was made to implement K-Means using **Hadoop Streaming** with a `mapper.py` and `reducer.py` pair. The mapper assigned each player to the nearest centroid and emitted `(cluster_id, features)` key-value pairs via stdout; the reducer computed new centroids from stdin grouped input. While the map and reduce logic was functionally correct, the Hadoop Streaming approach presented significant operational friction:

- Managing the iterative nature of K-Means (re-running the job per iteration, broadcasting updated centroids between jobs) required manual orchestration outside Hadoop, making the workflow fragile and difficult to reproduce.
- Performance overhead from launching a new JVM-based Hadoop job for each of the 20 iterations was prohibitive.
- Debugging streaming jobs was considerably harder than working within an interactive PySpark session.

The approach was ultimately abandoned in favor of the PySpark RDD-based implementation (`kmeans_spark.py`), which preserves the identical map–reduce logic but within a single, iterative, fault-tolerant Spark application.

8.2 Market Value Regression Model

An attempt was made to build a **regression model to predict log(market value)** from player, club, injury, and performance features. This was outlined in the project proposal as part of the predictive analysis. However, the model was not included in the final delivery due to challenges with the market value table: market valuations are recorded at irregular intervals and are not aligned to seasonal boundaries, making it difficult to construct a clean, leakage-free feature

matrix. The significant number of players with zero or missing market values further complicated target variable definition. This remains a strong candidate for future work with additional data cleaning effort.

---

9. Results Summary

Component	Method	Key Metric
Player Clustering	K-Means MapReduce (PySpark, K=5)	Silhouette = 0.45
Transfer Prediction – Model 1	KNN (PySpark ML, K=3)	Test Accuracy = 0.646
Transfer Prediction – Model 2	KNN MapReduce from scratch (K=3)	Test Accuracy = 0.630
Transfer Pattern Analysis	Heatmaps (5 / 10 / 15 years)	Descriptive
Agent–Club Analysis	Co-occurrence heatmaps	Descriptive
Dual Citizenship Analysis	Co-occurrence heatmaps	Descriptive
Nationality Co-occurrence	Heatmaps (with/without same-country)	Descriptive

---

10. Enhancements & Future Work

- **Address class imbalance** in the transfer prediction models using SMOTE oversampling or class-weighted loss functions. The current ~72/28 imbalance causes poor recall on the Transfer class.
- **Market value regression** using gradient-boosted trees on player-season aligned valuations, once the temporal alignment challenge is resolved.
- **Formal association rule mining** using FP-Growth on grouped league-season transfers to discover statistically significant transfer corridors.
- **Fully distributed deployment** on a multi-machine Spark cluster (or AWS EMR / Azure HDInsight) to process the full 5.7M record dataset without subsampling.
- **Graph-based agent network analysis** to uncover communities of agents who operate together across clubs and leagues.

---

11. Code & Notebooks

File	Description
`EDA.ipynb`	Full exploratory data analysis across all tables
`transfer_pattern.ipynb`	Transfer frequency heatmaps by year, age, and position
`agents_clubs_realtionship.ipynb`	Agent–club co-occurrence analysis
`citizenshiped_players.ipynb`	Dual citizenship heatmaps
`nationality_pairs.ipynb`	Nationality co-occurrence among teammates
`player-categorizing-model.ipynb`	K-Means clustering pipeline and results
`transfer-window-prediction-model.ipynb`	KNN Models 1 & 2 for transfer prediction
`kmeans_spark.py`	Standalone PySpark K-Means MapReduce implementation